Simulador Virtual de Cajero Automático (ATM)

Documentación del proyecto · Proyecto final · Prácticas Profesionalizantes I

1. Introducción

El proyecto es un simulador de cajero automático pensado para que personas mayores de edad, principalmente de la tercera edad, practiquen cómo usar un cajero en un entorno sin riesgo: no se maneja dinero real, no hace falta una cuenta bancaria y un error no tiene consecuencias. La aplicación imita el aspecto y el comportamiento de un cajero real, con pantalla y botones laterales, teclado numérico, ranura de tarjeta, bandeja de billetes e impresora de comprobantes, y guía al usuario con mensajes claros.

Está desarrollada solo con HTML, CSS y JavaScript, sin librerías ni servidor: se abre con doble clic en `index.html` desde cualquier navegador. El equipo trabajó sobre un repositorio Git y con un documento de requerimientos funcionales (carpeta `docs/Requerimientos-funcionales`) que también sirvió para repartir las tareas.

Para probarla se usa una cuenta simulada: clave `1234` y saldo inicial de $ 150.000,00.

2. Funcionalidades

La aplicación cubre las operaciones más habituales de un cajero real:

Funcionalidad	Qué hace el usuario	Reglas principales
Inicio de sesión	Inserta la tarjeta e ingresa la clave de 4 dígitos con el teclado numérico	Tiene 3 intentos. Al tercer error se avisa y se expulsa la tarjeta
Consultas	Ve su saldo y los últimos movimientos	Se muestran los 4 movimientos más recientes: hay datos simulados al inicio y se suman las operaciones hechas durante la sesión
Extraer efectivo	Elige uno de 4 montos fijos u otro monto	Múltiplo de $ 100, máximo $ 50.000 por extracción, con efectivo en el cajero y saldo suficiente. Los billetes salen en la ranura y se retiran con un botón

Funcionalidad	Qué hace el usuario	Reglas principales
Transferencias	Transfiere por CBU o por alias y consulta el historial	CBU de 22 dígitos, alias de tres palabras separadas por puntos y saldo suficiente
Pago de servicios	Paga servicios, recarga celular o SUBE y consulta pagos y vencimientos	El importe se descuenta del saldo. Los vencimientos son datos simulados
Cambio de clave	Ingresa la clave actual, la nueva y la repite	La nueva debe tener 4 números y ser distinta de la actual. Tres errores en la clave actual expulsan la tarjeta
Comprobante	Imprime el ticket de la última operación	El ticket sale en el módulo COMPROBANTE del cajero y gasta una hoja. Si no hay papel, se avisa
Salir	Retira la tarjeta y puede comenzar otra sesión	Se vacían la ranura de billetes y la impresora

Pensando en el público de la aplicación, hay un aviso fijo ("Está practicando: este cajero es un simulador, no se usa dinero real"), textos y botones grandes, mensajes de error que explican qué pasó y qué hacer, y botones de pantalla alineados con las teclas del bisel, como en un cajero verdadero.

3. Estructura del proyecto

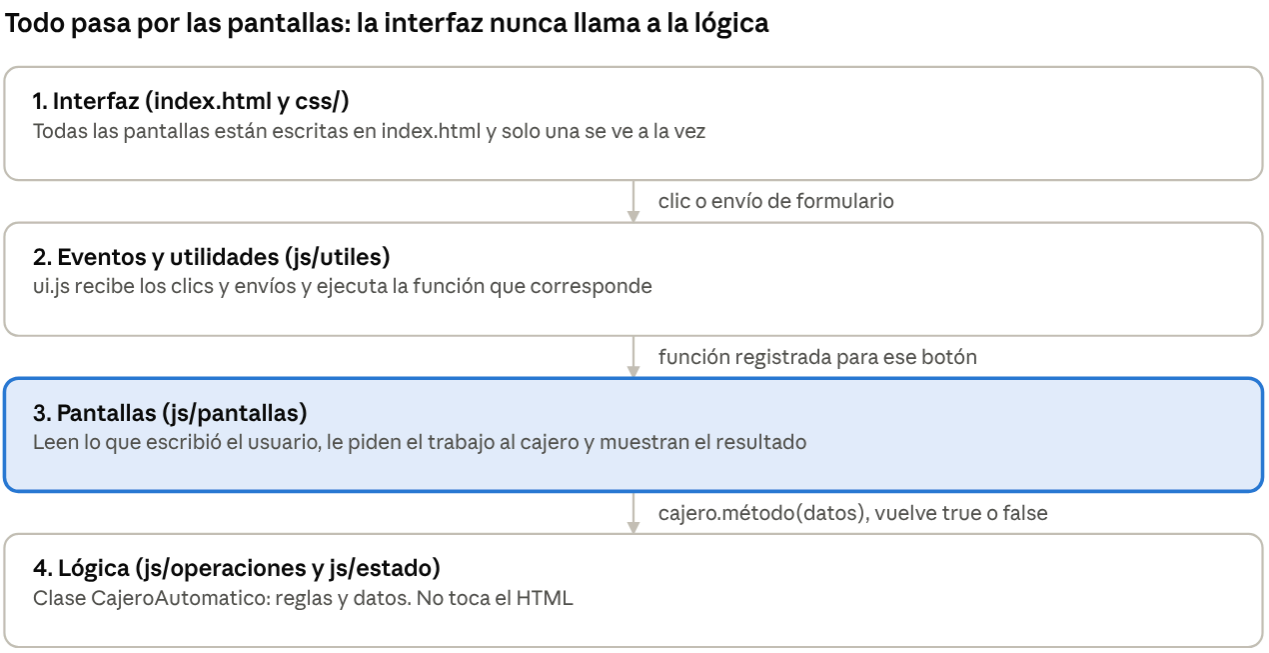
```
Proyecto/
├── index.html          Única página: contiene todas las pantallas
├── css/                base · layout · pantallas · componentes ·
accesibilidad
├── js/
|   ├── main.js         Crea el cajero y arranca la aplicación
|   ├── config/         constantes.js · datos-simulados.js
|   ├── estado/         estado.js (clase CajeroAutomatico y sus propiedades)
|   ├── operaciones/    Lógica: tarjeta, autenticación, consultas,
extracción,
|   |                   transferencias, servicios, claves, comprobante
|   ├── pantallas/      Una función por pantalla y el manejo de sus
formularios
|   └── utiles/         ui.js · teclado.js · formato.js · validaciones.js
├── assets/             Imágenes, fuentes y audio
├── docs/               Requerimientos, diagramas, capturas e informe
└── tests/              Pruebas automáticas y casos de prueba manuales
```

`index.html` carga los scripts en un orden que respeta las dependencias: configuración, estado, utilidades, pantallas, operaciones y, al final, `main.js`. Se usan scripts clásicos y no módulos de JavaScript, porque los módulos exigen un servidor y el proyecto debe poder abrirse con doble clic.

4. Cómo funciona el código

4.1 Arquitectura por capas

El código se organiza en cuatro capas con una regla simple: cada capa solo habla con la vecina. La interfaz (HTML y CSS) nunca llama directamente a la lógica. Los eventos del usuario los recibe `ui.js`, que ejecuta la función de pantalla correspondiente, y solo las pantallas le piden trabajo al objeto `cajero`. Ese objeto no toca el HTML: responde `true` o `false` y deja el motivo del rechazo en la propiedad `motivoRechazo`.



arquitectura por capas · 4 capas, las pantallas en el medio

Las pantallas son la única capa que conoce a la vez la interfaz y la lógica. Por eso un cambio visual no rompe las reglas del cajero, y una regla nueva no obliga a tocar el HTML.

4.2 Un solo HTML: pantallas que se muestran y se ocultan

`index.html` contiene todas las pantallas escritas de antemano, cada una como un bloque con su propio `id` (`pantallaLogin`, `pantallaInicio`, `pantallaExtraer`, etc.). Solo una está visible: las demás llevan el atributo `hidden`. Cambiar de pantalla es una sola llamada, `mostrarPantalla("pantallaLogin")`, que oculta las demás, limpia los formularios, dibuja las listas dinámicas si la pantalla las tiene y actualiza el saldo y los botones del pie. Las funciones `mostrarPantallaLogin()`, `mostrarPantallaMenuPrincipal()` y sus hermanas son atajos de una línea que llaman a `mostrarPantalla` con el id correspondiente.

4.3 Cómo se conectan los botones con las funciones

No hay un `onclick` en cada botón. Un único listener sobre `document`, creado en `conectarEventos()`, atiende todos los clics y envíos de formulario y decide según un atributo del HTML. Cada pantalla anota sus funciones en tres registros (objetos declarados en `ui.js`), de modo que `ui.js` no necesita conocer a las pantallas.

Atributo en el HTML	Cuándo actúa	Qué hace
`data-destino="pantallaPagos"`	Al hacer clic en el botón	Muestra esa pantalla con `mostrarPantalla`

Atributo en el HTML	Cuándo actúa	Qué hace
`data-accion="consultarSaldo"`	Al hacer clic en el botón	Llama a la función registrada con ese nombre en `accionesBoton`
`data-operacion="transferirCbu"` (en un formulario)	Al enviar el formulario	Llama a la función registrada en `manejadoresFormulario`
Clase `teclaFisica`	Al hacer clic en el teclado numérico	Escribe en el campo activo; CORREGIR, CANCELAR y CONFIRMAR tienen su propio comportamiento
`renderizadoresPantalla`	Al mostrar una pantalla	Dibuja las listas dinámicas (historial, movimientos, vencimientos)

Agregar un botón nuevo alcanza con dos pasos: poner el atributo en el HTML y registrar su función, por ejemplo `accionesBoton.miAccion = miFuncion;`.

4.4 La clase CajeroAutomatico

Toda la lógica vive en una clase. Sus propiedades están en `estado.js`. Las siete que definió el equipo en el documento de requerimientos son `estadoTarjeta`, `sesionActiva`, `estadoDispensador`, `efectivoDisponibleCajero`, `saldoDisponibleCuentaActual`, `claveUsuario` y `cantidadPapelDisponible`. Se sumaron propiedades de apoyo: `intentosPin`, `motivoRechazo`, los historiales, `movimientos`, `ultimaOperacion`, `billetesEntregados` y `comprobanteEmitido`.

JavaScript no permite partir una clase en varios archivos, así que los métodos se agregan desde cada archivo de `operaciones/` con `Object.assign(CajeroAutomatico.prototype, {...})`. Para el resto del código es como si estuvieran todos dentro de la clase: `cajero.transferirDineroCBU(...)`. Cada método valida, modifica el estado y devuelve `true` o `false`. Si rechaza la operación, explica por qué en `motivoRechazo`.

4.5 Ejemplo: una transferencia por CBU

1. El usuario escribe el CBU y el monto y toca "Confirmar". El navegador dispara el evento `submit` del formulario.
2. El listener lee el atributo `data-operacion` y ejecuta `procesarTransferenciaCbu(formulario)`, la función registrada para ese formulario.
3. La pantalla lee los campos y llama a `cajero.transferirDineroCBU(monto, cbu)`.

4. El cajero valida la sesión, el monto, el saldo y que el CBU tenga 22 dígitos. Si todo está bien, descuenta el saldo y registra la operación en el historial, en los movimientos y como última operación (la que usa el comprobante).
5. Si devolvió `false`, la pantalla muestra `motivoRechazo` bajo el formulario y no cambia de pantalla. Si devolvió `true`, muestra la pantalla de resultado con el saldo nuevo.

Los pagos, las recargas y el cambio de clave siguen el mismo patrón.

4.6 Extracción de efectivo con espera

Es la única operación con un paso asíncrono. Al elegir un monto, la pantalla le pide a `cajero.validarExtraccion(monto)` que revise las reglas. Si pasan, muestra "Contando billetes..." y programa con `setTimeout` la extracción real para 1,5 segundos después. Recién entonces `cajero.extraerEfectivo(monto)` descuenta el saldo y el efectivo del cajero y reparte el monto en billetes de mayor a menor valor con `dispensarBilletes`; la pantalla los muestra en la ranura "Retire su dinero". Si el usuario presiona CANCELAR durante la espera, la extracción no se ejecuta, así que nunca queda dinero descontado sin entregar.

5. Decisiones de diseño

1. **Un solo HTML con pantallas ocultas.** Se comparó con un HTML por pantalla y con generar todo desde JavaScript. Varios HTML obligan a repetir el marco del cajero en cada archivo y pierden el estado (saldo, sesión) en cada cambio de página. Generar todo con JavaScript mezcla el HTML dentro de textos y dificulta que cada integrante trabaje su parte. Con un solo HTML el estado vive en memoria, el marco existe una vez y HTML, CSS y JS quedan separados. Solo las partes que dependen de los datos (listas, mensajes) se generan desde JavaScript.
2. **Lógica separada de la presentación.** La clase `CajeroAutomatico` nunca toca el DOM. Esto permite probarla sin navegador, cambiar el diseño sin riesgo de romper reglas y repartir el trabajo: una persona en lo visual y otra en la lógica.
3. **Una clase repartida por archivos.** Cada archivo de `operaciones/` agrupa una funcionalidad (extracción, claves, comprobante). Eso ayuda a encontrar el código y reduce los conflictos de Git cuando varias personas trabajan a la vez.
4. **Un único listener y tres registros.** Agregar un botón no obliga a tocar el listener ni la navegación.
5. **Datos simulados y estado en memoria.** Cada práctica empieza de cero, sin base de datos ni persistencia, lo que es adecuado para un simulador de aprendizaje. Los datos de prueba están en `datos-simulados.js` y las reglas numéricas (límite de extracción,

intentos de clave, tiempo de espera) en `constantes.js`, así se cambian en un solo lugar.

6. **Sin librerías ni servidor.** Con scripts clásicos y rutas relativas, el proyecto funciona al abrir `index.html`. Es importante para el público destinatario y para entregarlo y defenderlo sin instalar nada.
7. **Pensado para adultos mayores.** Aviso permanente de práctica, textos grandes, mensajes que explican el problema y confirmaciones visibles (billetes en la ranura, ticket en la impresora). Los botones de pantalla están alineados con las teclas del bisel para que la práctica se parezca a la realidad. El comprobante se dibuja dentro del cajero y no en una ventana emergente, que el navegador podría bloquear.
8. **Convenciones de código.** El README fija camelCase, 4 espacios, comillas dobles, punto y coma y líneas de hasta 100 caracteres, para que el código de todo el equipo se lea igual.

6. Pruebas

La lógica se prueba con 41 pruebas automáticas que se ejecutan con `node tests/ejecutar-tests.js`. Cubren la autenticación, la extracción y el reparto de billetes, las transferencias, los pagos, el cambio de clave, los movimientos y el comprobante. Se verificó que detectan errores rompiendo el código a propósito y comprobando que la prueba correspondiente fallaba.

La parte visual se prueba con 22 casos manuales descritos en `tests/casos-de-prueba/casos-manuales.md`, cada uno con sus pasos y su resultado esperado.

7. Limitaciones y mejoras posibles

- **Sin persistencia:** al recargar la página, el saldo y el historial vuelven a su valor inicial. Podría guardarse el estado con `localStorage`.
- **Logos remotos:** los logos de la pantalla se cargan desde internet. Conviene descargarlos a `assets/img/logo/` para que la aplicación funcione sin conexión.
- **Un único usuario simulado:** hay una sola cuenta de prueba. Podrían ofrecerse varios perfiles con distintos saldos para practicar casos como el saldo insuficiente.
- **Accesibilidad:** a futuro podrían agregarse guía por voz, tamaño de texto ajustable y modo de alto contraste.
- **Más operaciones:** depósitos, más servicios y recargas, o un modo de práctica guiada con consignas paso a paso.

Anexo: diagramas del repositorio

Los diagramas completos están en `docs/diagramas/` (formato SVG, y PNG en la subcarpeta `png/`):

- Arquitectura por capas.
- Clase CajeroAutomatico: propiedades y métodos por archivo.
- Flujo de pantallas.
- Secuencia de una transferencia por CBU.
- Secuencia de una extracción de efectivo.
- Cómo se conectan los botones con las funciones.